AgileFlow AI

Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation

File 10 - Reusable Component and UI Integration Plan

Frontend component system, Figma-to-React mapping, UI reuse rules and implementation handoff guidance

Document ID	File 10
Audience	Frontend implementation team, UI/Figma team, group leader, AI coding agents
Purpose	Define reusable frontend components and show how Figma designs must be converted into consistent React implementation.
Development Stack	React + Vite + Tailwind CSS frontend; FastAPI backend; MongoDB knowledge base

1. Purpose of This Document

This document defines the reusable frontend component system for AgileFlow AI. It explains how the Figma designs should be converted into React components while keeping a single visual style, predictable folder structure, and reusable UI patterns across all agent pages.

- Prevent duplicate components and inconsistent UI styles.
- Give the implementation team a clear component library before page development starts.
- Keep the dark purple theme, approval gates, workflow status, traceability, and report screens consistent.
- Allow AI coding agents to work safely by giving them component boundaries and do-not-duplicate rules.

2. Core UI Integration Principle

Principle	Rule
One design system	All pages must use the centralized theme tokens from File 09. Do not hardcode random colors, shadows, or spacing values.
One component source	If a UI pattern appears more than once, create it as a reusable component instead of rebuilding it in each page.
One workflow language	Use official component names such as Requirement Gathering and Analysis Agent, Project Management and Design Agent, Implementation and Git Automation Agent, and Testing, Validation and Security Agent.
One state model	All pages must use the shared workflow and approval statuses:

	PENDING, IN_PROGRESS, WAITING_FOR_APPROVAL, APPROVED, REJECTED, FAILED, COMPLETED.
One integration path	Figma design -> shared component -> page composition -> API service -> backend response -> workflow state update.

3. Recommended Frontend Component Architecture

Use a layered component structure. Start with generic UI components, then build domain components for AgileFlow AI, then compose full pages.

```
frontend/src/
├── components/
│   ├── layout/
│   ├── ui/
│   ├── forms/
│   ├── data-display/
│   ├── workflow/
│   ├── agents/
│   └── reports/
├── pages/
├── services/
├── styles/
├── constants/
├── hooks/
└── utils/
```

Layer	Purpose	Examples
Layout components	Control application shell and page-level structure.	MainLayout, Sidebar, Topbar, PageContainer
Base UI components	Small reusable primitives used everywhere.	Button, Card, Modal, StatusBadge, Tabs
Form components	Reusable form and input patterns.	TextInput, TextArea, SelectField, FileUploadBox
Data display components	Reusable ways to show lists, tables, logs, and timelines.	DataTable, LogViewer, ActivityTimeline
Workflow components	Reusable SDLC workflow and approval controls.	WorkflowStepper, ApprovalGateCard, StageStatusCard
Agent components	Domain-specific cards used in agent pages.	AgentSummaryCard, AgentOutputPanel, ArtifactList
Report components	Reusable final evidence and traceability UI.	TraceabilityTable, ReportCard, ExportPanel

4. Reusable Layout Components

Component	Location	Used By	Purpose
MainLayout	components/layout/MainLayout.jsx	All authenticated pages	Wraps sidebar, topbar, main content area and consistent page spacing.

Sidebar	components/layout/Sidebar.jsx	MainLayout	Primary navigation for Dashboard, Projects, agents, Orchestrator, Traceability and Reports.
Topbar	components/layout/Topbar.jsx	MainLayout	Shows project selector, search placeholder, notifications and user profile area.
PageContainer	components/layout/PageContainer.jsx	All pages	Controls max width, responsive padding and section spacing.
PageHeader	components/layout/PageHeader.jsx	All pages	Standard title, subtitle, breadcrumb and page-level action buttons.
SectionHeader	components/layout/SectionHeader.jsx	Dashboard sections and cards	Small reusable heading for page sections.

Rule: No agent page should create its own sidebar, topbar, page header or page spacing. Always use the shared layout components.

5. Base UI Components

Component	Main Props / Variants	Usage Rule
Button	variant: primary, secondary, ghost, danger; size: sm, md, lg	Use for all actions. Do not use plain HTML buttons directly inside pages.
Card	variant: default, elevated, bordered, gradient-subtle	Use for dashboard cards, agent panels and report summaries.
StatusBadge	status: PENDING, IN_PROGRESS, APPROVED, FAILED, COMPLETED	Use same badge for workflow, agent, test and approval status.
Modal	title, children, confirmAction, cancelAction	Use for approval confirmation, reject reason and delete/clear actions.
Tabs	items, activeKey, onChange	Use for artifact groups such as Requirements, User Stories, API, DB, Tests and Reports.
AlertBox	type: info, success, warning, error	Use for workflow warnings, missing inputs, failed generation and validation messages.
EmptyState	title, description, actionLabel	Use when no project, artifact, test case, bug, or report exists yet.
LoadingState	label, size	Use during API calls and generation actions.
ErrorState	message, retryAction	Use for failed API calls or unsupported actions.

6. Workflow and Approval Components

AgileFlow AI depends on human approval gates. These components must be reusable and consistent across H1 to H5.

Component	Purpose	Where Used

WorkflowStepper	Displays the full SDLC stage sequence and current stage.	Dashboard, Project Workflow page, Orchestrator page
StageStatusCard	Shows one workflow stage with status, owner, latest output and next action.	Project Workflow page
ApprovalGateCard	Shows approval gate details, checklist, comments and approve/reject/request changes buttons.	H1, H2, H3, H4, H5 pages
ApprovalHistory	Shows reviewer, decision, comment and timestamp history.	All approval screens
NextActionPanel	Shows what the user should do next in the workflow.	Dashboard and Project Workflow page
FailureRouteCard	Shows issue source, target agent and correction action.	Orchestrator and Testing/Security pages

```
// Example approval gate props
<ApprovalGateCard
  gate="H2"
  title="Design and Planning Approval"
  status="WAITING_FOR_APPROVAL"
  artifactSummary={designSummary}
  onApprove={handleApprove}
  onReject={handleReject}
  onRequestChanges={handleRequestChanges}
/>
```

7. Agent-Specific Reusable Components

Component	Purpose	Used In
AgentSummaryCard	Shows agent name, responsibility, input count, output count and current status.	All agent pages
AgentInputPanel	Shows source artifacts received by the current agent.	Requirement, Design, Testing, Implementation pages
AgentOutputPanel	Displays generated artifacts with review actions.	All agent output pages
ArtifactList	Reusable list for requirements, designs, tests, code modules and reports.	All agent pages
ArtifactPreview	Shows selected artifact content in a clean preview area.	Agent pages and Report pages
QualityIssuePanel	Shows ambiguity, conflict, missing information, bugs or security findings.	Requirement and Testing/Security pages
GenerationActionPanel	Holds Generate, Regenerate, Save Draft and Send for Approval actions.	Requirement, Design, Test Planning and Implementation pages

8. Form and Input Components

Component	Purpose	Validation / UX Rule
TextInput	Single-line fields such as project name, domain, branch name.	Always show label and error message area.
TextArea	Client idea, reviewer comments, clarification answers.	Use character-friendly height and keep resize vertical only.
SelectField	Status, priority, artifact type, agent type.	Use fixed options from constants, not hardcoded page lists.
FileUploadBox	Upload documents, audio, video or generated reports.	Show accepted types, max size and uploaded file list.
SearchInput	Search artifacts, requirements, reports or logs.	Use consistent search width and placeholder.
FilterBar	Reusable filter row for status, type, priority, agent and date.	Do not duplicate filters inside each table.
CommentBox	Approval comments and feedback to agents.	Required when rejecting or requesting changes.

9. Data Display and Report Components

Component	Purpose	Important Columns / Data
DataTable	Reusable table for projects, requirements, backlog, tests, bugs and reports.	Supports title, columns, rows, loading, empty, action column.
TraceabilityTable	Shows end-to-end traceability.	Requirement ID, Design ID, Test ID, Code ID, Bug/Security ID, Status.
ReportCard	Shows final report summary.	Report type, generated date, status, export action.
MetricCard	Shows dashboard and validation metrics.	Total projects, approvals, passed tests, bugs, security findings.
ActivityTimeline	Shows workflow event history.	Time, actor/agent, event type, message.
LogViewer	Shows build logs, runtime logs, test logs and security scan logs.	Monospace text, severity filter, copy/download action.
ExportPanel	Export reports and matrices.	PDF, DOCX, CSV buttons; disabled until report exists.

10. Page-to-Component Mapping

Page	Required Components	Main Data Source
Dashboard	MainLayout, PageHeader, MetricCard, WorkflowStepper, DataTable, ActivityTimeline	projects, workflow_events

Create Project	PageHeader, TextInput, TextArea, FileUploadBox, Button, AlertBox	projects, client_inputs
Project Workflow	WorkflowStepper, StageStatusCard, ApprovalGateCard, NextActionPanel	workflow, approvals
Requirement Gathering and Analysis	AgentSummaryCard, AgentInputPanel, AgentOutputPanel, QualityIssuePanel, ApprovalGateCard	requirements, approvals
Project Management and Design	ArtifactList, DataTable, Tabs, TraceabilityTable, ApprovalGateCard	design_artifacts, backlog_items
Pre-Code Test Planning	ArtifactList, DataTable, TraceabilityTable, ApprovalGateCard	test_cases, traceability_links
Implementation and Git Automation	ArtifactList, LogViewer, ActivityTimeline, ApprovalGateCard	implementation_artifacts, execution_logs
Testing, Validation and Security	MetricCard, DataTable, LogViewer, ReportCard, ApprovalGateCard	test_results, bug_reports, security_findings
Orchestrator	WorkflowStepper, FailureRouteCard, ActivityTimeline, DataTable	workflow_events, route_failures
Final Reports	ReportCard, TraceabilityTable, ExportPanel	reports, traceability_links

11. Figma-to-React Integration Rules

1. Break every Figma screen into reusable sections before coding the page.
2. Identify repeated patterns such as cards, tables, badges, approval panels and tabs.
3. Create the reusable component first, then compose the page using that component.
4. Use theme tokens from File 09. Do not copy random Figma color values unless they match the theme tokens.
5. Use the same component names in Figma and React where possible.
6. Keep page components thin. Business logic, API calls and data mapping should be separated into services/hooks.
7. When Figma changes, update the component once rather than manually editing every page.

Figma Layer / Frame	React Output	Example
Frame: Dashboard	Page component	pages/dashboard/DashboardPage.jsx
Component: Status Badge	Reusable UI component	components/ui/StatusBadge.jsx
Component: Approval Gate Card	Workflow component	components/workflow/ApprovalGateCard.jsx
Table block	Data display component	components/data-display/DataTable.jsx
Report section	Report component	components/reports/ReportCard.jsx

12. Component Naming Rules

Item Type	Naming Rule	Example
Component file	PascalCase.jsx	ApprovalGateCard.jsx
Page file	PascalCase + Page.jsx	ProjectWorkflowPage.jsx

Service file	camelCase + Service.js	projectService.js
Hook file	use + PascalCase.js	useProjectWorkflow.js
Constants file	camelCase.js	workflowStatus.js
CSS module/token file	kebab-case or token name	tokens.css

- Do not create names like Button2, NewCard, TestFinalCard or MyComponent.
- Do not create duplicate components with similar names such as StatusLabel and StatusBadge. Use the approved component name.
- Keep official system component labels readable in UI; avoid only using Agent 1, Agent 2, Agent 3 and Agent 4 as page titles.

13. API Integration Pattern for Components

Reusable components should not directly call backend APIs unless they are specifically designed as data containers. Prefer pages or hooks to call services, then pass clean data into components.

```
// Preferred pattern
Page -> custom hook -> service -> API client -> backend
Page -> passes data into reusable component

// Avoid
Reusable StatusBadge -> directly calls backend
DataTable -> directly knows project workflow API details
```

Layer	Responsibility
apiClient.js	Base URL, headers, JSON parsing and common error handling.
projectService.js	Project APIs such as create, list, details and workflow.
agent service files	Requirement, design, testing and implementation-specific API calls.
hooks	Loading, error, refresh and state mapping logic.
components	Visual rendering and user interaction callbacks only.

14. Component Ownership and Conflict Prevention

Area	Owner	Rule
Layout components	Group leader / foundation team	Only the foundation team should modify MainLayout, Sidebar and Topbar initially.
Base UI components	Frontend foundation owner	Changes need review because all pages depend on these components.
Agent page components	Assigned member/component owner	Members may build inside their own agent folders using shared components.
Workflow components	Group leader + orchestrator owner	Approval and workflow components should not be changed without approval.
Report and traceability components	Shared responsibility	Changes must preserve required traceability fields and report structure.

- If a component is useful for more than one page, move it to components/ instead of keeping it inside one page folder.

- If a component is used by only one agent page, it can stay inside that agent page folder at first.
- Do not edit shared components during agent feature work unless the PR clearly explains why.

15. Accessibility and Responsive UI Rules

- Every input must have a visible label and error state.
- Buttons must have clear action text such as Approve, Request Changes, Run Validation or Generate Tests.
- Do not rely only on color to show success, error or warning; also show text labels.
- Tables must remain readable on smaller screens using horizontal scroll if needed.
- All components should support keyboard navigation and visible focus states.
- Use sufficient contrast for dark purple backgrounds and light text.
- Long artifact content and logs should use scrollable content areas inside cards, not break the whole page layout.

16. Initial Component Build Order

8. Create theme tokens and global CSS setup using File 09.
9. Create MainLayout, Sidebar, Topbar, PageContainer and PageHeader.
10. Create Button, Card, StatusBadge, AlertBox, LoadingState, ErrorState and EmptyState.
11. Create form components: TextInput, TextArea, SelectField and FileUploadBox.
12. Create data components: DataTable, MetricCard, ActivityTimeline and LogViewer.
13. Create workflow components: WorkflowStepper, StageStatusCard and ApprovalGateCard.
14. Create agent components: AgentSummaryCard, AgentInputPanel and AgentOutputPanel.
15. Create report components: ReportCard, TraceabilityTable and ExportPanel.
16. Build pages by composing reusable components, not by manually recreating sections.

17. AI Coding Agent Prompt Template for UI Components

Use this prompt format when asking an AI coding agent to create or update reusable frontend components.

```
You are working on AgileFlow AI frontend.

Task:
Create/update [component name].

Allowed folder:
frontend/src/components/[folder-name]/

Do not modify:
- backend/
- frontend/src/services/ unless required
- frontend/src/styles/ unless the task is theme-related
- existing shared components unless explicitly requested

Follow these documents:
- File 09: Frontend Implementation and Centralized Theme Guide
- File 10: Reusable Component and UI Integration Plan

Requirements:
- Use centralized theme tokens.
- Use existing Button, Card, StatusBadge and layout components when available.
- Do not duplicate existing components.
- Add loading, empty and error states where needed.
- Keep props clear and reusable.
- Return changed files and short test instructions.
```

18. Pull Request Checklist for UI Components

Checklist Item	Required?
Component uses centralized theme tokens, not random hardcoded colors.	Yes
Component is reusable and not tightly coupled to one hardcoded project.	Yes
Loading, empty and error states are handled when applicable.	Yes
Component name follows agreed naming convention.	Yes
No duplicate component was created.	Yes
Page uses shared layout and page header.	Yes
API calls are placed in service/hook layer, not inside small visual components.	Yes
Screenshots or short UI evidence added to PR when visual UI changed.	Yes
No unrelated files were modified.	Yes

19. Final Acceptance Criteria

- The frontend has one shared layout and navigation structure.
- All pages use the same dark purple theme and centralized tokens.
- Reusable components are created before full page implementation.
- Approval gates, workflow states and traceability sections use shared components.
- Agent pages look consistent even though different members implemented them.
- AI coding agents can safely work using clear allowed folders and do-not-change rules.
- The final UI is suitable for a research prototype demo and clearly explains AgileFlow AI workflow.

Appendix A - Recommended Component Inventory

Category	Components
Layout	MainLayout, Sidebar, Topbar, PageContainer, PageHeader, SectionHeader
Base UI	Button, Card, Modal, Tabs, StatusBadge, AlertBox, Tooltip, Divider
Forms	TextInput, TextArea, SelectField, Checkbox, FileUploadBox, CommentBox, FilterBar
Data Display	DataTable, MetricCard, ActivityTimeline, LogViewer, ArtifactPreview
Workflow	WorkflowStepper, StageStatusCard, ApprovalGateCard, ApprovalHistory, NextActionPanel, FailureRouteCard
Agents	AgentSummaryCard, AgentInputPanel, AgentOutputPanel, ArtifactList, QualityIssuePanel, GenerationActionPanel
Reports	ReportCard, TraceabilityTable, ExportPanel, ValidationSummary, EvidencePanel

This inventory should be created gradually. Do not block implementation by building every component perfectly at once. Build the most repeated components first, then improve them during page implementation.